This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2026.3653260

A Hybrid Explainable AI for DDoS Attacks Detection in Industrial IoT Networks

Makhduma F Saiyed, *Member, IEEE*, Irfan Al-Anbagi, *Senior Member, IEEE*

***Abstract*—The Industrial Internet of Things (IIoT) has transformed industrial processes by allowing real-time data monitoring and automation. While IIoT offers many operational advantages, it is still at risk of cyberattacks like Distributed Denial of Service (DDoS) attacks. This paper introduces a novel Hybrid Explainable AI for DDoS Attack Detection (HEAD) system. The HEAD system combines deep learning, feature attribution, and model optimization within IIoT networks. Unlike earlier systems that rely on tree-based or black-box models without post-training optimization, the HEAD system introduces three main innovations. Firstly, it applies a combined explanation-based feature selection approach using SHAP and LIME to identify both global and local feature importance. The geometric mean of the normalized SHAP and LIME values is used to rank and select the most informative features. Secondly, the HEAD system uses a feedforward deep neural network trained on the selected features to learn important traffic patterns. It maintains high detection accuracy while keeping the model simple and efficient. This replaces traditional ML models with a more compact and flexible architecture. Thirdly, the system applies SHAP GradientExplainer to the trained model to identify neurons in the first hidden layer that are influenced by the input features. These neurons are pruned to create a lightweight version of the model that reduces computation without compromising accuracy. Evaluation on the HL-IoT, ToNIoT, and Edge-IIoTSet datasets shows that the HEAD system attains over 94% accuracy and enhances model efficiency through SHAP-guided pruning. To support real-world deployment the HEAD system provides an interactive interface that shows SHAP visualizations and training results. This helps network administrators monitor predictions and improve detection policies. This design ensures that HEAD is not only accurate but also transparent and suitable for deployment in real IIoT networks.**

***Index Terms*—Explainable AI (XAI), Feature Engineering, Human-Computer Interaction, High- and Low-Volume DDoS attacks, IoT Security, LIME, Machine Learning, SHAP**

I. Introduction

The Industrial Internet of Things (IIoT) has improved industrial processes by connecting physical devices with digital systems. It provides real-time data analysis, smart management, and proactive maintenance [1], [2]. IIoT also plays a key role in sectors such as manufacturing, logistics, healthcare, agriculture, supply chains, and energy. It helps increase efficiency and reduce operational costs [3]. IIoT minimizes the need for human involvement and simplifies operations through real-time data collection, analysis, and automation [4]. It also supports autonomous decision-making in industrial environments. Moreover, the use of various communication protocols in IIoT systems improved global connectivity. It helps enterprises link industrial operations with cloud and edge computing to build scalable and intelligent solutions [5].

Securing networks is a major challenge in the advancement of IIoT systems [6], [7]. IIoT networks handle large amounts of sensitive data that are transmitted to the edges or clouds, increasing their vulnerability to cyberattacks [8]. Among the various attacks in IIoT systems, Distributed Denial of Service (DDoS) attacks are serious because they can disrupt critical services by flooding network resources with malicious traffic [9], [10]. DDoS attacks can be classified into two main types: high-volume attacks and low-volume attacks [11], [12]. High-volume attacks involve massive floods of traffic that can overwhelm networks in seconds. Low-volume attacks include Slowloris and stealthy HTTP floods. They use slow traffic patterns to avoid detection [13]. This dual behaviour of DDoS attacks creates specific challenges for IIoT security systems. They must be able to detect both large-scale disruptions and subtle, low-rate attacks at the same time [14].

Detection of high- and low-volume DDoS attacks in IIoT networks is based on the use of Machine Learning (ML) and Deep Learning (DL) models [4], [15], [16]. These models are trained to recognize patterns that separate normal traffic from malicious activity. However, their effectiveness depends on the quality of the input features used during training. To improve detection performance, researchers have proposed various feature engineering methods such as Genetic Algorithms (GA), Random Forest (RF), and optimization-based methods [9], [17]. Recent research has applied Explainable AI (XAI) methods such as SHapley Additive exPlanations (SHAP) and Local Interpretable Model-Agnostic Explanations (LIME) to identify key features. These methods aim to improve both explainability and detection performance [18], [19]. These methods improve model clarity and detection accuracy. However, they require high computation and are often too slow for real-time deployment. Moreover, many existing systems are inaccurate in detecting both high- and low-volume DDoS attacks. This highlights the need for fast, lightweight, and explainable detection frameworks for IIoT networks [20].

Explainable Artificial Intelligence (XAI) has become a key area of research to address the interpretability challenges of black-box ML models in IIoT networks [21], [22]. Techniques such as SHAP and LIME have not been fully explored to provide global and local explanations for model predictions in IIoT networks [23], [24]. SHAP provides consistent and additive feature importance scores, which makes it useful for

Makhduma F Saiyed is with the Department of Computer Science, Trent University, Oshawa, ON L1J 5Y1, Canada. (Email: makhdumabanu-saiyed@trentu.ca)

Irfan Al-Anbagi is with the Faculty of Engineering and Applied Science, University of Regina, Regina, SK S4S 0A2, Canada. (Email: Irfan.Al-Anbagi@uregina.ca)

Authorized licensed use limited to: University of Waterloo. Downloaded on March 19,2026 at 17:04:48 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2026.3653260

detecting critical features in high- and low-volume DDoS attacks. Batchu et al. [25], [26] applied SHAP-based recursive feature elimination to improve model transparency, but did not consider effective Human-Computer Interaction (HCI) mechanisms. Without HCI, network administrators find it difficult to interpret model outputs and refine detection strategies. Sharma et al. [27] combined SHAP and LIME to explain DL decisions but did not consider the need for interactive user interfaces. The interactive user interface is necessary for real-time network management and informed decision-making. The lack of HCI-focused solutions limits the practical usefulness of these systems in operational environments [20], [28].

Existing work faces several challenges in building unified systems for effective and explainable DDoS detection in IIoT networks. One major challenge is the separation of explainability and optimization. Most approaches apply XAI tools like SHAP and LIME only after model training. They do not integrate these tools into model design to guide optimization steps such as neuron pruning. Another challenge is the limited use of combined explainability techniques. SHAP and LIME are often used independently, thereby missing opportunities to use their strengths for more reliable feature selection. Additionally, some existing frameworks include visual tools to explain model behaviour, but these are often complex or limited to specific domains. As a result, non-expert users find them difficult to understand and apply effectively. To overcome these challenges, this paper introduces the Hybrid Explainable AI for DDoS Detection (HEAD) system. It addresses key challenges such as the high computational cost of feature selection and the difficulty in detecting both high and low-volume attacks. It also handles the limited explainability of ML and DL models and the lack of user-friendly solutions for practical deployment. The HEAD system is designed to detect both high- and low-volume DDoS attacks in IIoT networks. It uses hybrid feature engineering alongside XAI methods such as SHAP and LIME. Explainability is built into the feature selection and model optimization process rather than added after training. SHAP identifies the overall importance of each feature, and LIME explains individual predictions. Together, they make the learning and decision processes more transparent. The HEAD system uses SHAP-based pruning to remove less essential neurons, making the DNN more efficient and easier to understand. In addition, the system features an interactive HCI design that allows network administrators to visualize attack patterns and feature importance. This improves usability and supports better decision-making. The key contributions of this paper are as follows:

- The HEAD system proposes a computationally efficient hybrid feature selection method that combines SHAP and LIME scores from a shallow Random Forest model. By applying XAI to a small data sample and using simple geometric fusion, the system reduces the number of evaluations and avoids complex aggregation. This keeps the process lightweight while still offering feature-level explainability.
- The HEAD system designs a lightweight DNN and applies a novel SHAP-based pruning method after training to identify and remove low-importance neurons. This reduces computations during inference, reduces resource demands, and supports deployment in IIoT networks without affecting accuracy.
- The HEAD system integrates SHAP and LIME for interpretability, providing global insights into model behaviour and local explanations for individual predictions. This supports transparency and helps administrators understand the model's decisions.
- The HEAD system develops a user interface that allows administrators to visualize attack patterns, the importance of features, and the prediction of the system. The visualizations improve usability and support more effective feature engineering and model adjustments.
- The HEAD system evaluates its performance using multiple datasets, including HL-IoT [9], an IoT dataset with high- and low-volume DDoS attacks, and Edge-IIoTSet [29] and ToN-IoT [30], which contain high-volume attack scenarios in IoT and IIoT networks.

The remainder of the paper is organized as follows: Section II reviews the related work, Section III presents the proposed HEAD system, Section IV explains the implementation of the testbed and performance evaluation, and Section V concludes the paper.

II. Related Work

The existing literature focuses on feature engineering and ML/DL techniques to detect botnet activities, DDoS attacks, and other malicious behaviours in IoT and IIoT networks [4], [33]–[36]. There has been research on ML and DL-based DDoS detection [13], [37]. However, only a few works specifically target high- and low-volume DDoS attacks using advanced techniques [9], [38]. XAI has also gained attention to improve the interpretability of black-box models [23], [39], [40], but most studies use XAI only for post-hoc analysis. While XAI is often used for post-hoc interpretation, there is no existing work that uses XAI for model optimization, such as pruning, to improve efficiency. Although HCI has been explored separately, its integration with XAI for real-time visualization and interaction in DDoS detection systems remains limited.

A. Feature Engineering and ML-Based Detection in IoT/IIoT Networks

Several recent studies focus on feature engineering techniques and advanced detection algorithms. Their goal is to improve performance and resource efficiency in IIoT networks. These techniques focus on optimizing input features and model accuracy, but often struggle with scalability and explainability challenges. Taher et al. [18] presented an ML-based approach to detect botnet attacks in IIoT environments. Their model used a hybrid feature selection method combining filter and wrapper techniques with the Grasshopper Optimization Algorithm (GOA) and k-Nearest Neighbours (kNN) to identify key features. An Improved Harris Hawks Optimization (IHHO) algorithm was then used to fine-tune neural network parameters and improve detection accuracy. The N-BaIoT dataset was

Authorized licensed use limited to: University of Waterloo. Downloaded on March 19,2026 at 17:04:48 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2026.3653260

TABLE I: Comparison of Existing Approaches for DDoS Detection in IIoT Networks

Reference	Dataset	Feature Engineering Method	High/Low-Volume Attack	Explainability	HCI Integration	Computational Efficiency
[24]	Simulated and real environments	k-dimensional tree	✓	✓	✓	Moderate
[31]	UNSW-NB15 and custom dataset	Pearson Correlation Coefficient	X	X	X	Moderate
[27]	NSL-KDD and UNSW-NB15	SHAP, LIME	X	✓	X	Moderate
[18]	N-BaIoT	Clustering, Grasshopper Algorithm	X	X	X	Low
[19]	UNSW-NB15	Genetic Algorithm	X	X	X	Low
[1]	CICDDoS2019	XGBoost	✓	X	X	High
[32]	UNSW-NB15, CPS	Independent Component Analysis	✓	X	X	Low
[25]	CICDDoS2019	SHAP-based RFE	✓	✓	X	Moderate
[17]	IoT-CIDDS	Statistical analysis, Random Forest	X	X	X	Low
[33]	KDDCUP-99, UNSW-NB15	NA	X	X	X	low
[16]	Edge-IIoTset	NA	X	X	X	low
[9]	HL-IoT, ToN-IoT	T-test and Genetic Algorithm	✓	X	X	Moderate
[13]	HL-IoT, ToN-IoT, CICIDS, ISCX, HICT	Zeek	✓	X	X	Moderate
HEAD	HL-IoT, EdgeIIoT, ToN-IoT	SHAP and LIME	✓	✓	✓	High

used to test multiclass botnet attack detection. However, the complexity of feature selection and the limited adaptability to evolving attacks created challenges. Kasongo [19] developed an intrusion detection system for IIoT networks that used a GA for feature selection alongside classification models such as Random Forest (RF) and Extra Trees (ET). Although the system achieved strong results, it required time-consuming feature engineering and high computational resources. It also lacked explainability features and user interaction through HCI. Malik et al. [17] proposed a feature engineering and ML framework for DDoS detection in IoT networks using the IoT-CIDDS dataset. The paper focused on feature selection and dimensionality reduction to improve the efficiency of the ML model. It evaluated several classifiers based on accuracy, precision, recall, F1-score, and computational time. Although feature reduction improved intrusion detection, the process was time-intensive and less suitable for dynamic networks. Additionally, the framework did not address high- and low-volume DDoS attacks. Shafin et al. [41] presented an explainable feature selection framework for detecting phishing on the web using ML. The framework, called SHAP-LIME Aggregation Feature Selection (SLA-FS), used SHAP for global interpretability and LIME for local analysis to identify important features for each class. The Web Page Phishing Dataset was used for evaluation, where SLA-FS reduced 87 features to 56. Performance tests were conducted with RF, XGBoost (XGB), and kNN models. However, the feature selection process showed a need for further improvements, such as adding ensemble stacking, to handle the changing nature of phishing attacks. Saiyed and Al-Anbagi [9] proposed the Genetic Algorithm and t-Test for DDoS Attack Detection (GADAD) system to detect high- and low-volume DDoS attacks in IoT networks. The GADAD system used a feature selection method called GAStats, which combined GA with statistical evaluation. Tree-based ML models, including RF and ET, were used for classification. Although the system performed efficiently, it needed further optimization for real-world deployment to manage scalability and dynamic attack patterns. The above-mentioned systems improved detection accuracy. However, they lacked the adaptability, transparency, and efficient real-time operation needed for modern IIoT networks.

B. Advanced DL and Optimization-Based Intrusion Detection Frameworks

DL and optimization-based frameworks have been introduced to capture complex attack patterns and improve generalization across IIoT scenarios. These methods aim to improve adaptability and detection accuracy. They also try to control computational overhead in limited-resource environments. Several studies have utilized DL architectures to improve DDoS detection in IIoT and SDN environments. Zainudin et al. [1] presented a DDoS detection and classification system for software-defined IIoT networks using a hybrid CNN-LSTM (Convolutional Neural Network–Long Short Term Memory) model. The system used XGBoost for feature selection to identify the most relevant features to reduce dimensionality and improve efficiency. When tested on the CICDDoS2019 dataset, it achieved good accuracy with low computational complexity. However, it lacked HCI features for better interpretability and user interaction, and showed limited adaptability to evolving DDoS attack patterns. Pooja and Shrinivasacharya [33] developed a Bi-LSTM-based intrusion detection model tested on the KDDCUP-99 and UNSW-NB15 datasets. The model achieved up to 99.7% accuracy. It handled complex sequential network traffic through dropout and time-distributed layers. However, it lacked real-time detection and online deployment, which was noted as future work. Yoon and Kim [42] introduced an intrusion detection system for Software-Defined Networking (SDN). Their approach used a Divide and Conquer Attention (DCA) mechanism to improve the detection of Distributed Denial of Service (DDoS) attacks in SDN networks. The DCA mechanism focused on learning attack patterns and prioritizing critical network traffic features to improve anomaly detection. The system was tested in virtual network attack scenarios. The evaluation used the Open

Authorized licensed use limited to: University of Waterloo. Downloaded on March 19,2026 at 17:04:48 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2026.3653260

Network Operating System (ONOS) controller and the Mininet simulator. However, it relied heavily on synthetic SDN datasets and struggled to adapt to rapidly evolving attack behaviours.

Other works have explored hybrid and optimization-based frameworks. These approaches aim to improve the efficiency and adaptability of intrusion detection in IIoT environments. Gyamfi and Jurcut [31] developed a customized intrusion detection system for IIoT networks. It combined an online incremental support vector data description method to detect anomalies in IIoT devices with an Adaptive Sequential Extreme Learning Machine (AS-ELM). The AS-ELM ran on Multi-access Edge Computing (MEC) servers to optimize computational resources. The system was evaluated using the UNSW-NB15 dataset and a custom-generated dataset to detect and classify different types of network intrusion. However, the system had some limitations, such as dependence on specific datasets, high computational demands, and challenges with real-time performance. Wei et al. [43] proposed a hybrid intrusion detection model. It integrates autoencoders for feature extraction with a neural network for classification. This approach addresses the issue of limited sample sizes in industrial control systems. The model used an ensemble of shallow and deep autoencoders to capture both linear and nonlinear patterns. It then applied a neural network to reduce false positives. While the method improved detection in limited data scenarios, it lacked XAI-driven interpretability and user interface support. Mao et al. [44] proposed a hierarchical federated class-incremental learning model for intrusion detection in IIoT networks. The system learned to recognize new attack classes over time without forgetting previous ones. It used local training and global aggregation across edge devices to achieve this. The model was tested on the NSL-KDD and ToN-IoT datasets. However, the system relied heavily on federated communication and faced challenges in handling concept drift in dynamic IIoT environments. Similarly, Moustafa et al. [32] presented a threat intelligence framework to improve cybersecurity in Industry 4.0 environments. This approach used a beta Mixture-Hidden Markov model (MHMM) to identify cyber threats in physical and network systems. The framework combined data management with feature reduction techniques, using Independent Component Analysis (ICA) for feature selection. It was validated on both a power system dataset and the UNSW-NB15 dataset. However, the system had high resource requirements, which limited its applicability in resource-constrained environments. Deep models such as CNN-LSTM and Bi-LSTM improved detection rates. However, they incurred high computational costs and lacked explainability or interactive user integration. Similarly, optimization-based approaches improved adaptability but remained limited by computational complexity and the absence of integrated explainability.

C. Explainable AI and Human–Computer Interaction in DDoS Detection

XAI is now used to explain black-box DDoS detection models, and some systems include basic Human–Computer Interaction (HCI) features for visualization. Feng et al. [24] presented a kNN-based learning framework to detect DDoS attacks, which was tested in both real and simulated environments. The system was adaptable and could operate across different network configurations without retraining. It also used a k-dimensional tree to improve efficiency. A notable aspect of the framework was its strong focus on explainability. This strengthened HCI by integrating AI-generated insights directly into the user interface. However, their work did not address computational overhead. Similarly, Sharma et al. [27] developed an explainable ML-based intrusion detection system for IoT networks using DNN and CNN models. These models were tested on the NSL-KDD and UNSW-NB15 datasets. XAI methods such as LIME and SHAP were then used to explain the decision-making process. However, their system did not address important issues such as class imbalance in the datasets or detection of low-volume attacks. Batchu and Seetha [25] proposed an integrated framework for detecting DDoS attacks in IoT networks. The system used SHAP-based recursive feature elimination (RFE) for feature selection and dynamic ensemble selection techniques (KNORA-E and KNORA-U) for classification. The model provided both global and local explanations to improve transparency. However, the system had limitations, including high computational overhead due to the use of dynamic ensemble techniques and a lack of focus on HCI capabilities. The XAI-based works mentioned above focus mainly on post-hoc analysis. They do not use explanations to optimize models or improve computational efficiency. Few recent works in other domains have explored attribution-based pruning to make neural networks more compact and interpretable. For example, relevance-based pruning of CNNs and transformers has been shown to improve both efficiency and explainability without degrading accuracy [45], [46]. However, these techniques have not been applied to DDoS detection or IIoT security models. Table I provides a summary of ML and DL approaches for DDoS detection in IIoT networks. It highlights their ability to manage different attack types, provide explainability, include HCI for better usability, and maintain computational efficiency.

Although AI-based DDoS detection has progressed in IIoT networks, many existing systems are still difficult to deploy due to high computational demands [47], [48]. DL models often demand significant processing power, and while XAI methods like SHAP and LIME improve interpretability, they are computationally heavy. As a result, these methods are less practical for resource-constrained edge networks [21], [49]. Most current approaches apply XAI after model training and do not incorporate it into optimization processes. No existing work uses XAI to guide neuron pruning, which could simplify models and speed up inference without reducing transparency. Furthermore, existing HCI-integrated systems often lack interactive visualization for administrators [48]. These limitations affect scalability, responsiveness, and operational trust in real-world deployments. To overcome these limitations, the proposed HEAD system uses a compact Feedforward DNN with SHAP-informed neuron pruning. This reduces model size and improves inference efficiency. A hybrid SHAP-LIME feature selection method improves the quality and relevance of the explanation while minimizing redundancy. Although XAI

Authorized licensed use limited to: University of Waterloo. Downloaded on March 19,2026 at 17:04:48 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2026.3653260

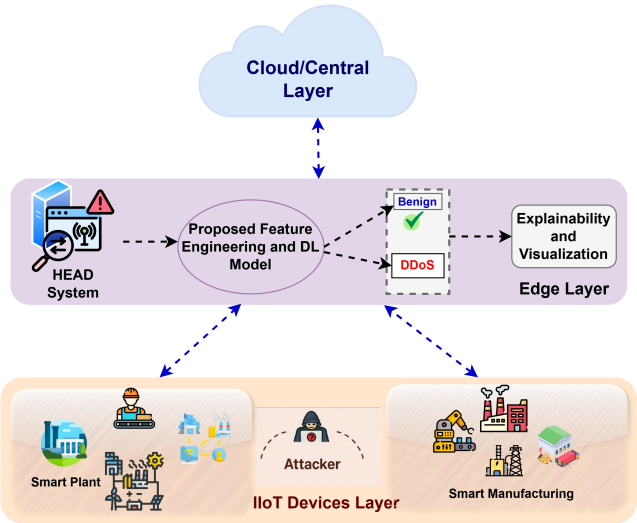


Fig. 1: Architecture of the HEAD System

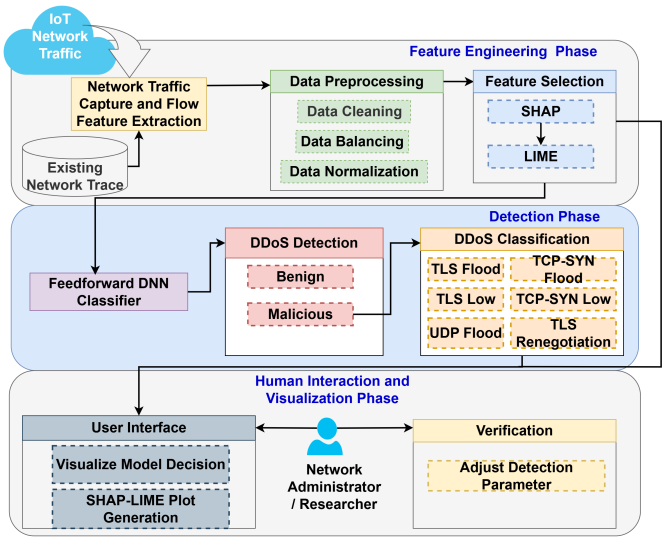


Fig. 2: Phases of the HEAD System

methods add some computational overhead, their use is limited to offline training and refinement stages. The HEAD system also provides an interactive interface that visualizes important features. This gives decision-makers clear and actionable insights.

III. Proposed DDoS Attack Detection System: HEAD

The HEAD system architecture is designed with three interconnected layers to detect and analyze DDoS attacks in IIoT networks, as shown in Fig. 1. The first layer, called the IIoT Devices Layer, represents the operational environment that includes smart plants and smart manufacturing systems. These devices generate legitimate traffic essential for operations, but are also vulnerable to cyberattacks. All network traffic, including normal and malicious data, is sent to the next layer for inspection and analysis. At the Edge Layer, the proposed HEAD system is deployed to perform real-time DDoS detection and interpretation. This layer handles traffic monitoring, hybrid feature engineering, and DL-based classification using a Feedforward DNN model. The model, structured with multiple dense layers, is trained on refined features to classify traffic accurately into different DDoS categories. SHAP and LIME are used to explain model predictions by providing both global and local feature importance. The SHAP GradientExplainer helps identify neurons with minimal contribution. These neurons are pruned to reduce complexity and improve inference speed. The processed traffic is then categorized as benign or DDoS. The Edge Layer also supports visualization through an interactive interface that displays model results. This helps administrators fine-tune detection parameters and respond quickly to potential threats. The final layer, the Cloud/Central Layer, serves as a coordination and storage hub. It collects logs, summarized detection results, and model updates from multiple edges to support centralized analysis and scalability in large IIoT networks. The cloud layer also performs periodic retraining of the HEAD model when new attack patterns are detected.

The HEAD system operates in three main phases, as shown in Fig. 2. In the Feature Engineering Phase, raw network traffic data or existing network traces are collected and processed using a Zeek-based parser. This includes cleaning inconsistent values, balancing class distributions, and normalizing feature scales. SHAP and LIME are applied together to select the most relevant features by combining global importance with local refinement. This ensures that the extracted patterns accurately represent both normal and attack behaviours. In the Detection Phase, the refined features are fed into a feedforward DNN model deployed at the edge. The model detects and classifies DDoS attacks by learning the statistical and behavioural patterns of network flows. It distinguishes between benign and malicious traffic and identifies different types of DDoS attacks, such as TCP-SYN Flood, UDP Flood, ICMP Flood, HTTP Flood, TLS Flood, and TLS Renegotiation attacks. These attacks differ in packet size, rate, and protocol-level behaviour, allowing the model to learn unique temporal and volumetric characteristics for each. By modelling these variations, the HEAD system detects both high-volume attacks that overwhelm bandwidth and low-volume attacks that exploit protocol vulnerabilities. Finally, in the Human Interaction and Visualization Phase, the system provides a user interface to visualize model predictions and generate SHAP–LIME explanations. In addition to examining how specific features affect each classification, administrators can also review the SHAP-LIME-selected features. This helps them understand which features had the greatest influence during training. This phase bridges automation with human decision-making, ensuring transparency and improving response strategies during DDoS attacks. The subsequent sections provide a detailed explanation of each phase.

A. Feature Engineering Phase

The feature engineering phase is an important component of the HEAD system. This phase converts raw network traffic

Authorized licensed use limited to: University of Waterloo. Downloaded on March 19,2026 at 17:04:48 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2026.3653260

Algorithm 1 SHAP-LIME Feature Engineering

Require: Dataset $X \in \mathbb{R}^{N \times M}$, labels y, model f, SHAP fraction α, LIME fraction β, threshold τ
Ensure: Final selected feature set K_{Final}
$X \leftarrow \text{Clean}(X)$
$X \leftarrow \dfrac{X - \min(X)}{\max(X) - \min(X)}$
// Compute SHAP importance for all M features
for each feature $i = 1$ to M **do**
  Compute ϕ_i using the SHAP equation
  SHAP Importance$(i) = \dfrac{1}{n} \sum_{k=1}^{n} |\phi_{i,k}|$
end for
// Select top SHAP features
Sort features by SHAP importance
$K_{\text{SHAP}} =$ indices of the top $\lceil \alpha M \rceil$ features
// Compute LIME only on SHAP-selected features
for each sample x in a subset of X **do**
  Compute local LIME scores for features in K_{SHAP}
end for
Compute LIME Importance(i) for each $i \in K_{\text{SHAP}}$
// Select top LIME features
Sort K_{SHAP} by LIME importance
$K_{\text{LIME}} =$ indices of top $\lceil \beta |K_{\text{SHAP}}| \rceil$ features
// Combine SHAP and LIME scores
for each feature $i \in K_{\text{LIME}}$ **do**
  CI$(i) = \sqrt{\text{SHAP Importance}(i) \times \text{LIME Importance}(i)}$
end for
// Apply CI threshold
$K_{\text{Final}} = \{i \in K_{\text{LIME}} \mid \text{CI}(i) \geq \tau\}$
return K_{Final}

data into optimized feature sets for DDoS detection. This phase starts with preprocessing, where the raw data is cleaned to address inconsistencies, such as missing or infinite values, which are replaced with zeros.

$$X = \text{Clean}(X), \tag{1}$$

where X shows the dataset. Following this, data normalization is applied to scale feature values within a defined range, which ensures comparability across features.

$$X = \frac{X - \min(X)}{\max(X) - \min(X)}. \tag{2}$$

To address class imbalances, the Synthetic Minority Over-sampling Technique (SMOTE) generates synthetic samples for underrepresented classes, resulting in a balanced dataset suitable for training. After preprocessing, feature selection is performed using SHAP and LIME.

SHAP calculates the global importance of features by assigning SHAP values to each feature i, using the equation:

$$\phi_i = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|!(|N| - |S| - 1)!}{|N|!} \left[f(S \cup \{i\}) - f(S) \right], \tag{3}$$

where ϕ_i is the SHAP value of the feature i, S is a subset of features that excludes i, N is the total set of features and $f(S)$ is the prediction of the model based on the subset S. The global importance of features is calculated by aggregating SHAP values across all samples:

$$\text{SHAP Importance}(i) = \frac{1}{n} \sum_{k=1}^{n} |\phi_{i,k}|, \tag{4}$$

where n is the total number of samples. Based on the global importance of SHAP, the main features K_{SHAP} are selected using:

$$K_{\text{SHAP}} = \lceil \alpha \times M \rceil, \tag{5}$$

where M is the total number of features and α represents the fraction of top features retained (e.g., $\alpha = 0.7$). In the HEAD system, $\alpha = 0.7$ is chosen to retain 70% of the most important features based on their SHAP values. This value is selected based on tests in which α is varied between 0.6 and 0.8 in increments of 0.1. The $\alpha = 0.7$ preserves a sufficient number of globally significant features to ensure high F1-scores while reducing the dimensionality of the features by 30%, which minimizes training time and complexity of the model without performance degradation. The choice of α is validated to ensure that the retained features effectively capture the critical characteristics of the dataset. These selected K_{SHAP} features are then passed to LIME for further refinement.

In LIME, the importance of local features is evaluated for individual predictions. For a given instance x, LIME approximates the model f with a locally linear surrogate model g by solving the following:

$$g(z') = \arg\min \left(L(f, g, \pi_x) + \Omega(g) \right), \tag{6}$$

where z' is a simplified representation of x, $L(f, g, \pi_x)$ is the loss function that measures the difference between f and g, π_x represents the proximity of the samples to x, and $\Omega(g)$ penalizes the complexity of g. LIME scores are computed for the K_{SHAP} features across a sample set and the features are ranked based on their average importance. The top K_{LIME} features are selected using:

$$K_{\text{LIME}} = \lceil \beta \times K_{\text{SHAP}} \rceil, \tag{7}$$

where β is the fraction of features retained (e.g., $\beta = 0.5$) after refinement with LIME. β determines how many of the K_{SHAP} features are considered important enough to be included in the next step of the pipeline. For example, if $\beta = 0.5$, it indicates that only 50% of the top features selected by SHAP are further refined by LIME. This value is chosen to balance the trade-off between retaining a sufficiently large set of features for robust model performance and reducing computational overhead by eliminating less critical features. The choice of β often depends on the characteristics of the dataset and the complexity of the problem; in our case, $\beta = 0.5$ is determined to ensure that the most informative features are retained without generating computational burden for the subsequent stages.

Finally, the SHAP and LIME importance scores for the features K_{LIME} are combined using the geometric mean to calculate the Combined Importance (CI):

$$\text{CI}(i) = \sqrt{\text{SHAP Importance}(i) \times \text{LIME Importance}(i)}. \tag{8}$$

Authorized licensed use limited to: University of Waterloo. Downloaded on March 19,2026 at 17:04:48 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2026.3653260

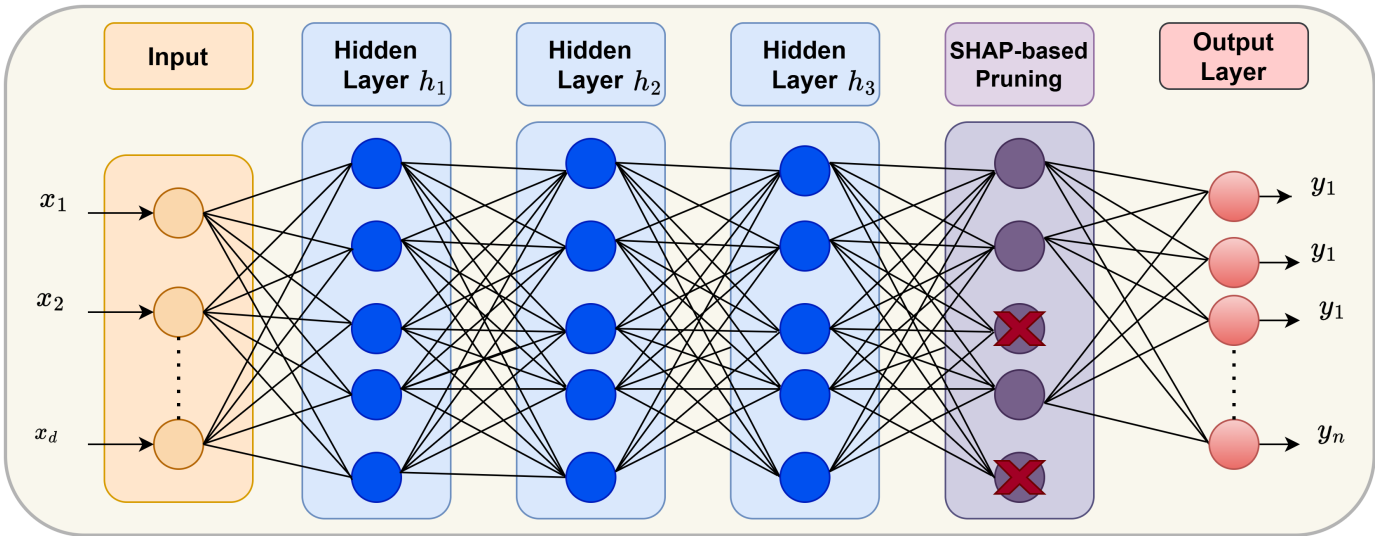


Fig. 3: Architecture of the Feedforward DNN Classifier with SHAP

Features with a CI score above a predefined threshold τ are selected as the final feature set:

$$K_{\text{Final}} = \{i \mid \text{CI}(i) \geq \tau\}, \tag{9}$$

where τ determines which features are included in the final feature set based on their CI score. In the HEAD system, τ is set to a small positive value, such as $\tau = 0.005$, to ensure that only features with a significant CI are selected. This threshold is selected by sweeping τ between 0.001 and 0.01 and evaluating its impact on performance and CI distribution. The value $\tau = 0.005$ filters out features with weak combined relevance while preserving those consistently ranked high by both SHAP and LIME. A lower threshold may include less important features, which can increase computational load. A higher threshold might exclude key features, leading to lower model performance. By setting $\tau = 0.005$, the system ensures that the final feature set includes only those features that are both globally important and locally significant. These selected features are then fed into the DL model for accurate DDoS detection.

In the HEAD system, SHAP and LIME work together to create an explainable feature selection process. SHAP gives a global view by showing how much each feature, like flow duration or packet rate, contributes on average to detection. LIME, on the other hand, explains individual predictions, showing why a specific flow is marked as benign or malicious. By combining both, the system gains a clear global and local understanding. This SHAP–LIME setup helps select features that are meaningful for security, not just random statistical patterns. The geometric mean is chosen for this fusion, as it balances SHAP's global contribution values with LIME's localized importance scores. Unlike the arithmetic mean, which can be dominated by large SHAP values, the geometric mean avoids this imbalance. It emphasizes features that remain consistently important across both explainers and reduces the influence of noisy or one-sided contributions [50]. Algorithm 1 summarizes the complete workflow used for the feature engineering step.

B. Detection Phase

The detection phase in the HEAD system is responsible for detecting and classifying DDoS attacks based on the refined features selected during the feature engineering phase. These features serve as input to a DL model designed using a feedforward DNN.

1) Architecture of the Feedforward DNN Classifier: The DNN classifier as shown in Fig. 3 comprises multiple fully connected layers that capture nonlinear relationships in the feature space. The architecture includes:

- An input layer of dimension d, where d is the number of selected features ($d = |K_{\text{Final}}|$).
- Hidden layers with h_1, h_2, and h_3 neurons, each followed by batch normalization and dropout to prevent overfitting.
- An output layer of size C, where C is the number of classes, with softmax activation for multi-class classification.

Mathematically, the forward pass through the network is defined as:

$$h^{(1)} = \text{ReLU}(W^{(1)}x + b^{(1)}), \tag{10}$$

$$h^{(2)} = \text{ReLU}(W^{(2)}h^{(1)} + b^{(2)}), \tag{11}$$

$$h^{(3)} = \text{ReLU}(W^{(3)}h^{(2)} + b^{(3)}), \tag{12}$$

$$\hat{y} = \text{Softmax}(W^{(4)}h^{(3)} + b^{(4)}), \tag{13}$$

where $x \in \mathbb{R}^d$ is the input feature vector, $W^{(l)}$ and $b^{(l)}$ are the weights and biases of the l-th layer, and $\hat{y}$ is the predicted class probability vector.

2) Loss Function and Training: To address class imbalance commonly present in DDoS datasets, the model is trained using focal loss:

$$\mathcal{L}_{\text{focal}} = -\sum_{i=1}^{C} \alpha_i (1 - p_i)^{\gamma} y_i \log(p_i), \tag{14}$$

where $\gamma > 0$ is the focus parameter, α_i is the balance weight of the class, p_i is the predicted probability of the class i, and y_i is the true label of the class (encoded as hot).

Authorized licensed use limited to: University of Waterloo. Downloaded on March 19,2026 at 17:04:48 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2026.3653260

3) SHAP-Based Neuron Pruning: Once the initial model is trained, the SHAP GradientExplainer is used to compute the contribution of each input neuron to the output. For each neuron j in the first hidden layer, the average SHAP value $\bar{\phi}_j$ across the validation samples n is calculated:

$$\bar{\phi}_j = \frac{1}{n} \sum_{k=1}^{n} |\phi_{j,k}|, \tag{15}$$

where $\phi_{j,k}$ is the SHAP value of the neuron j for sample k. Neurons with $\bar{\phi}_j < \delta$ are considered to have a negligible contribution and are pruned from the architecture:

$$P = \{j \mid \bar{\phi}_j < \delta\}, \tag{16}$$

where δ is a predefined pruning threshold (e.g., 10-70%). After pruning, the DNN is restructured by removing the corresponding neurons and retraining on the same data. This leads to reduced model size, faster inference time, lower memory consumption, and improved generalization performance.

Beyond improving efficiency, this pruning step also improves the interpretability of the neural network. In the HEAD system, neuron-level contribution scores are computed using a gradient-based SHAP attribution method. For each neuron in the first hidden layer, the average SHAP value shows how much it contributes to the model's output. Neurons with very low SHAP values are considered redundant and are removed, as shown in Fig. 3. This pruning eliminates neurons that contribute minimally to the decision process, resulting in a more compact and transparent network while maintaining predictive performance. As shown in the ablation study (Table V, Section IV-C), pruning up to 40% of the least influential neurons does not affect the F1-score across datasets, confirming that these neurons have limited impact on classification outcomes. Removing low-contribution neurons helps the model run faster and use less memory. It also improves generalization while maintaining accuracy.

4) Post-Pruning Retraining: The pruned network f_{pruned} is retrained with the same focal loss to fine-tune weights:

$$\min_{\theta} \mathcal{L}_{\text{focal}}(f_{\text{pruned}}(x;\theta), y), \tag{17}$$

where θ represents the new trainable parameters of the pruned model. The final model output is as follows:

$$\hat{y} = f_{\text{pruned}}(x), \quad x \in \mathbb{R}^{|K_{\text{Final}}|}. \tag{18}$$

This pruned and optimized model is then deployed for inference in the HEAD system, offering a balance between accuracy and efficiency. During detection, the model classifies incoming traffic as benign or malicious. If malicious traffic is detected, the system further categorizes the specific type of attack, including TLS flood, TCP-SYN flood, UDP flood, TCP flood, HTTP flood, ICMP flood, and TLS renegotiation. The classification is evaluated using three datasets: HL-IoT [9], ToN-IoT [30], and Edge-IIoTSet [29]. These data sets include various types of attack, as shown in Table II, along with their label.

C. Human Interaction and Visualization Phase

The human interaction and visualization phase allows network administrators and researchers to interpret, verify, and understand the decision-making process of the HEAD system. This phase focuses on explainability by translating the quantitative outputs of SHAP and LIME, defined in Section III-A, into visual and interpretable forms. The SHAP summary plot provides a global explanation by ranking features based on their average contribution across all samples. The same SHAP importance values ($\phi_{i,k}$) calculated during feature engineering are aggregated and displayed as a bar plot to show how each feature influences model predictions. The colour gradient shows how features like packet rate or flow duration affect attack probability. This help administrators spot overall network behaviour patterns. The LIME explanation chart offers a local explanation by showing how specific feature values affected an individual prediction. Using the same local surrogate models described earlier, the LIME visualization displays each feature's positive or negative contribution to the output for a given instance. This helps analysts understand why a particular session is labelled as benign or malicious and which parameters drove that decision.

Together, these SHAP and LIME visualizations provide both a global overview and a local rationale for every prediction, forming a complete explainability layer. The SHAP summary identifies the most influential network features overall, while LIME explains the reasoning behind each individual classification. Integrating these insights into a user-friendly dashboard lets administrators check model results, change decision limits, and review the system's logic in real time. This phase improves transparency, accountability, and user confidence. Every detection made by the HEAD system can be traced back to explainable and security-relevant evidence.

IV. Results and Evaluation

A. Experimental Datasets

1) Edge-IIoTSet Dataset: The Edge-IIoTSet dataset is generated using a seven-layer IoT/IIoT testbed to evaluate ML/DL-based intrusion detection systems. The dataset includes traffic from more than 10 IoT devices and supports various network protocols such as MQTT and Modbus. It

TABLE II: Categories and Label Encoding for Datasets

Category (Edge-IIoTSet)	Label	Category (HL-IoT)	Label	Category (ToN-IoT)	Label
DDoS_HTTP	0	Benign	0	TCP Flood	0
DDoS_ICMP	1	TLS-LoW-Cat1	1	UDP Flood	1
DDoS_TCP	2	TLS-LoW-Cat2	2	Benign	2
DDoS_UDP	3	TCP-SYN-Low	3		
Benign	4	TLSFlood	4		
		TLS-Pulses	5		
		TLS-Renegotiation	6		
		TCP-SYNFlood	7		

Authorized licensed use limited to: University of Waterloo. Downloaded on March 19,2026 at 17:04:48 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2026.3653260

contains 14 types of cyberattacks, categorized into five threat groups, including DoS/DDoS, injection, and malware attacks. The dataset uses 671,827 malicious samples and 249,429 benign samples. Training data is balanced, with 200,000 samples per category, covering benign traffic and four types of DDoS attacks (HTTP Flood, ICMP Flood, TCP Flood, and UDP Flood). The test dataset contains a total of 207,814 samples. Further details about this dataset are available in [29].

2) HL-IoT Dataset: The HL-IoT dataset [9] captures realistic IoT network traffic, including normal behaviour and high- and low-volume DDoS attacks. It includes 8 hours of benign traffic generated using protocols such as HTTP, MQTT, and custom application-layer protocols. High-volume attacks include a 25-minute TLS flood with excessive handshake requests [13] and a 15-minute TCP-SYN flood designed to exhaust the system resources [51]. Low-volume attacks include a 4-hour TCP-SYN flood with reduced packet rates and a 4-hour low-intensity TLS flood, both designed to evade detection. The dataset contains 295,194 benign flows and 748,677 DDoS flows. Training data is balanced with 200,000 samples per category, and the test set includes 45,210 samples.

3) ToN-IoT Dataset: The ToN-IoT dataset includes telemetry data from IoT and IIoT devices, operating system logs, and network traffic captured from a medium-sized IoT network. It includes labelled data for different network activities, including high-volume DDoS attacks such as UDP and TCP flooding. To keep the training dataset balanced, oversampling techniques are used. Each of the three categories, named benign, TCP flood, and UDP flood, contains 200,000 samples. In total, the training dataset consists of 600,000 normalized samples. A separate dataset with 45,043 samples is used to evaluate the system's performance. More details about this dataset are available in [30].

B. Experimental Setup and Evaluation Metrics

All experiments are conducted using Python 3.9 on Ubuntu 20.04, running on an Intel® Core™ i9-10900KF CPU @ 3.70 GHz with 64 GB RAM. TensorFlow 2.10 and scikit-learn 1.2 are used for model development and analysis, while SHAP 0.45 supports explainability. Each dataset is normalized to the range [0,1] using Min–Max scaling. Duplicate and incomplete flows are removed, and categorical attributes are label-encoded. The data are divided into 80% training and 20% testing sets using stratified sampling. To address class imbalance, RandomOverSampler is applied to minority classes, ensuring equal representation of each attack type. Class weights are also computed and passed during model training to handle residual imbalance. The classification model is a lightweight feedforward DNN. It begins with an input layer matching the number of selected features, followed by three fully connected layers with 64, 32, and 16 neurons, each using ReLU activation. A dropout layer with a rate of 0.2 is applied after the first dense layer to prevent overfitting. The output layer uses a softmax activation to support multiclass classification. The model is trained using the Adam optimizer with a learning rate of 0.001, a batch size of 256, and a maximum of 100 epochs. Early stopping is used to prevent overtraining. The loss function used is Sigmoid Focal Cross-Entropy with a focusing parameter $\gamma = 2.0$, which improves stability in imbalanced data scenarios. Explainability is integrated throughout the system using SHAP and LIME. SHAP provides global feature importance and neuron-level interpretability, while LIME offers local, instance-level explanations. Both are applied through a Random Forest baseline to maintain consistency between model interpretability stages. A Streamlit-based[1] interface visualizes SHAP summary plots and LIME explanations. This interface allows administrators to analyze attack patterns, feature importance, and prediction outcomes interactively.

TABLE III: Experimental Setup and Model Configuration

Parameter	Configuration
Operating System	Ubuntu 20.04 LTS
CPU	Intel Core i9-10900KF @ 3.70 GHz (20 threads)
Memory	64 GB RAM
Frameworks	TensorFlow 2.10, scikit-learn 1.2, SHAP 0.45
Data Split	80% Training / 20% Testing (Stratified)
Balancing Method	RandomOverSampler, Class Weights
Normalization	Min–Max scaling [0, 1]
Explainability Methods	SHAP Gradient Explainer, LIME Kernel Explainer
K_{Final}	26 features
Model Architecture	Dense(64)–Dense(32)–Dense(16)–Dense(8)
Optimizer	Adam
Loss Function	Sigmoid Focal Cross-Entropy ($\gamma = 2.0$)
Batch Size	256
Epochs	100 (with early stopping)
Dropout Rate	0.4
Interface	Streamlit Visualization Dashboard

Following model development and explanation, a two-stage optimization strategy is applied to improve system efficiency. In the first stage, hybrid feature selection is performed using SHAP and LIME explanations on a sampled subset of 100 instances. The importance scores from both methods are normalized and combined using geometric averaging to identify the most informative features. The top 30 features are selected, striking a balance between reduced dimensionality and maintained classification performance. The 100-instance subset is used only for computing SHAP and LIME importance scores. This keeps the computational cost low because both explainers are expensive to run on large datasets. The sample is selected with stratified random sampling so that the class distribution remains representative and avoids skewed importance scores. The goal here is to produce a stable importance ranking, not to generate explanations for every instance. SHAP and LIME importances are aggregated using mean absolute values. Prior work shows that these aggregated rankings remain stable even when computed on small but representative subsets, as long as the goal is feature ranking rather than instance-level explanation [52], [53]. To verify this, the same process is repeated with larger sample sizes up to 2000 instances. The selected top features remain consistent across these sizes, which shows that a 100-instance subset is enough to obtain

[1]https://streamlit.io/

Authorized licensed use limited to: University of Waterloo. Downloaded on March 19,2026 at 17:04:48 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2026.3653260

a stable ranking. The geometric mean then combines SHAP and LIME scores and reduces the influence of noise from either method. All later stages, including training, validation, pruning, and evaluation, use the full dataset, so this reduced subset does not introduce bias in the final model. In the second stage, the SHAP Gradient Explainer is used after initial training to analyze neuron-level contribution. Neurons in the lowest 40% percentile by SHAP value are pruned, and the model is retrained with the reduced architecture. This two-stage reduction, from feature selection to neuron pruning, makes the system efficient and suitable for IIoT environments with limited resources. System performance is measured using classification accuracy, precision (Pr), recall (Rc), F1-score (F1), feature processing time, and inference time. All experiments are repeated three times, and the mean values are reported to ensure reproducibility and statistical reliability. The classification process uses the attack types listed in Table II. The complete configuration is summarized in Table III.

C. Results and Analysis

The results of the HEAD system are evaluated in three main phases: network traffic preprocessing, hybrid feature selection, and user interaction and visualization. The computational time of the hybrid feature selection process, which uses SHAP and LIME, is measured to ensure that the set of features is efficiently refined before classification. An ablation study examines the effect of different SHAP pruning thresholds (δ) on model performance, showing that moderate pruning improves efficiency without compromising detection accuracy. The classification performance and inference time of the HEAD system are compared with various systems, including DEEPShield [13], Bi-LSTM [33], and LSTM [16], which shows its effectiveness in the detection of DDoS attacks. In the user interaction and visualization phase, SHAP and LIME plots explain the model's behaviour at both the feature and instance levels, helping administrators understand the hybrid model's decision-making process.

Fig. 4 shows the feature processing times of the SLA-FS [41] and HEAD systems on the HL-IoT, Edge-IIoT, and ToN-IoT datasets. The evaluation uses 100,000 samples per class and 10,000 samples for LIME analysis. The HEAD system processes features much faster than SLA-FS [41] across all datasets. On the HL-IoT dataset that contains 8 different classes, the HEAD system completes feature processing in 26,456 seconds, while SLA-FS [41] completes it in 31,876 seconds. On the Edge-IIoT dataset, the HEAD system processes features in 940 seconds compared to 1,856 seconds for SLA-FS [41]. For the ToN-IoT dataset, the HEAD system completes feature processing in 10,168 seconds, while SLA-FS [41] completes it in 14,226 seconds. The HEAD system is faster due to its two-phase feature selection method. First, SHAP is used to reduce the feature set, allowing LIME to process only the selected features. In contrast, SLA-FS [41] applies SHAP and LIME to all features, which increases processing time.

To evaluate the effectiveness of the SHAP–LIME fusion strategy, three aggregation functions are tested on the HL-IoT dataset: geometric mean, arithmetic mean, and a weighted sum. The weighted sum baseline uses equal contributions from SHAP and LIME after normalization. Table IV shows that the geometric mean gives the highest accuracy (0.94) and F1-score (0.96). The geometric mean improves the fusion because it highlights features supported by both SHAP and LIME. It also reduces the influence of features that seem important in only one method. The arithmetic mean treats both values additively, so features with moderate disagreement still receive relatively high combined scores, which leads to a slightly weaker feature set and lower performance. The weighted sum performs the worst because equal weighting cannot adjust for variations in the scale or reliability of the two explainers. These results confirm that the geometric mean produces a more stable and discriminative fusion of SHAP and LIME for the HEAD system.

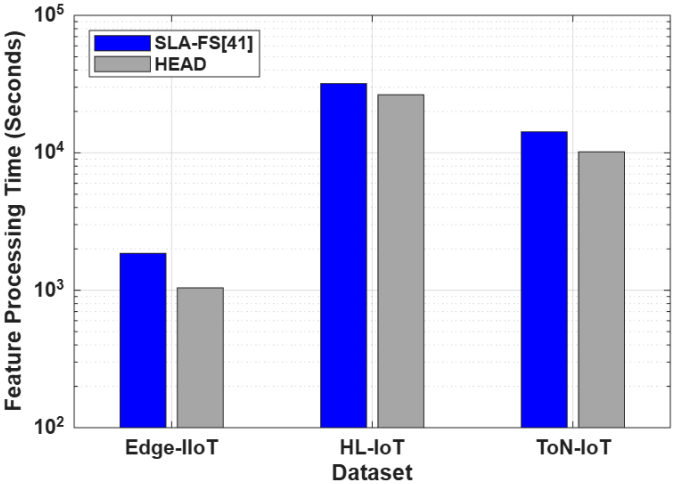


Fig. 4: Feature Processing Time Across Different Datasets

TABLE IV: SHAP-LIME Fusion Methods on HL-IoT dataset

Fusion Method	Accuracy	F1-Score
Geometric Mean	0.94	0.96
Arithmetic Mean	0.90	0.92
Weighted Sum	0.87	0.89

TABLE V: Ablation Study on SHAP-Based Pruning Thresholds (δ)

Dataset	Pruning (%)	10	20	30	40	50	60	70
Edge-IIoT	F1-Score	1.00	1.00	0.99	0.97	0.95	0.94	0.91
	Training Time (s)	20	20	19	19	19	18	18
HL-IoT	F1-Score	0.93	0.93	0.93	0.93	0.92	0.92	0.91
	Training Time (s)	85	85	87	86	86	86	86
ToN-IoT	F1-Score	0.98	0.94	0.97	0.98	0.88	0.86	0.85
	Training Time (s)	186	185	185	188	187	188	189

Table V shows the ablation study of SHAP-based pruning with different pruning percentages (δ) ranging from 10 to 70 for the Edge-IIoT, HL-IoT, and ToN-IoT datasets. In this experiment, the model retrains after removing the lowest $\delta\%$ of neurons based on their SHAP importance values, and the system records F1-score and training time for each case. The results show that the F1-score remains almost unchanged up

Authorized licensed use limited to: University of Waterloo. Downloaded on March 19,2026 at 17:04:48 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2026.3653260

TABLE VI: Performance Comparison of Different Systems on Datasets

Dataset	Class	LSTM [16]			Bi-LSTM [33]			DEEPShield [13]			1D-CNN [36]			HEAD			Pruned-HEAD		
		Pr	Rc	F1	Pr	Rc	F1	Pr	Rc	F1	Pr	Rc	F1	Pr	Rc	F1	Pr	Rc	F1
Edge-IIoT	0	0.92	0.88	0.90	0.89	0.86	0.87	0.93	0.95	0.94	0.91	0.90	0.90	0.97	0.98	0.97	0.97	0.96	0.96
	1	0.92	0.95	0.93	0.91	0.98	0.94	0.92	0.95	0.93	0.92	0.89	0.90	0.96	0.95	0.95	0.95	0.95	0.95
	2	0.97	0.92	0.94	0.94	0.92	0.93	0.94	0.94	0.94	0.94	0.91	0.92	0.99	0.96	0.97	0.98	0.97	0.97
	3	0.91	0.92	0.91	0.95	0.95	0.95	0.93	0.97	0.95	0.93	0.92	0.92	0.96	0.97	0.96	0.95	0.95	0.95
	4	0.90	0.86	0.88	0.89	0.88	0.88	0.91	0.91	0.91	0.87	0.88	0.87	0.95	0.97	0.96	0.97	0.97	0.97
HL-IoT	0	0.88	0.90	0.89	0.91	0.90	0.90	0.94	0.93	0.93	0.90	0.82	0.86	0.98	0.95	0.96	0.93	0.95	0.94
	1	0.92	0.90	0.91	0.89	0.88	0.88	0.93	0.93	0.93	0.81	0.75	0.78	0.98	0.98	0.98	0.97	0.99	0.98
	2	0.91	0.91	0.91	0.90	0.92	0.91	0.95	0.92	0.93	0.82	0.90	0.86	0.99	0.99	0.99	0.99	0.99	0.99
	3	0.92	0.93	0.92	0.92	0.94	0.93	0.94	0.94	0.94	0.91	0.92	0.91	0.95	0.94	0.94	0.95	0.91	0.93
	4	0.91	0.90	0.90	0.94	0.89	0.91	0.91	0.90	0.90	0.71	0.74	0.73	0.94	0.98	0.96	0.93	0.97	0.95
	5	0.94	0.93	0.93	0.93	0.91	0.92	0.93	0.97	0.95	0.74	0.73	0.73	0.97	0.96	0.96	0.95	0.93	0.94
	6	0.89	0.88	0.88	0.91	0.88	0.89	0.90	0.92	0.91	0.98	0.92	0.95	0.94	0.95	0.94	0.94	0.90	0.92
	7	0.91	0.92	0.91	0.89	0.93	0.91	0.92	0.93	0.92	0.91	0.91	0.91	0.97	0.96	0.96	0.96	0.95	0.95
ToN-IoT	0	0.87	0.90	0.88	0.88	0.89	0.88	0.93	0.92	0.92	0.93	0.95	0.94	0.99	0.98	0.98	0.99	0.97	0.98
	1	0.94	0.87	0.90	0.92	0.91	0.91	0.94	0.92	0.93	0.92	0.95	0.93	0.95	0.97	0.96	0.95	0.97	0.96
	2	0.92	0.89	0.90	0.87	0.91	0.89	0.90	0.91	0.90	0.92	0.90	0.91	0.91	0.96	0.93	0.91	0.93	0.92

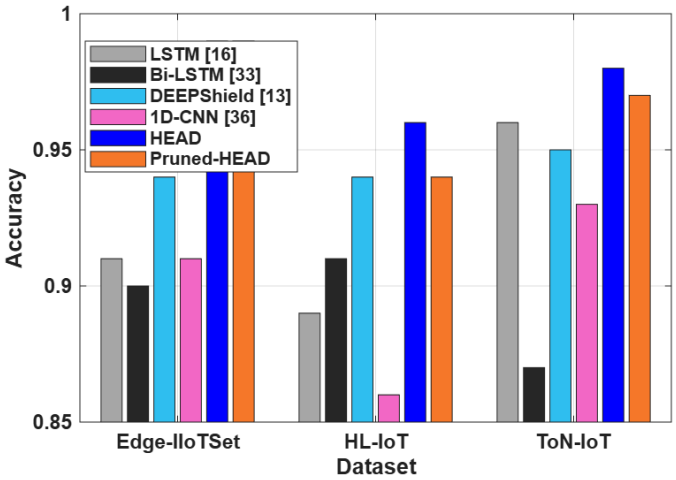


Fig. 5: System Accuracy Across Different Datasets

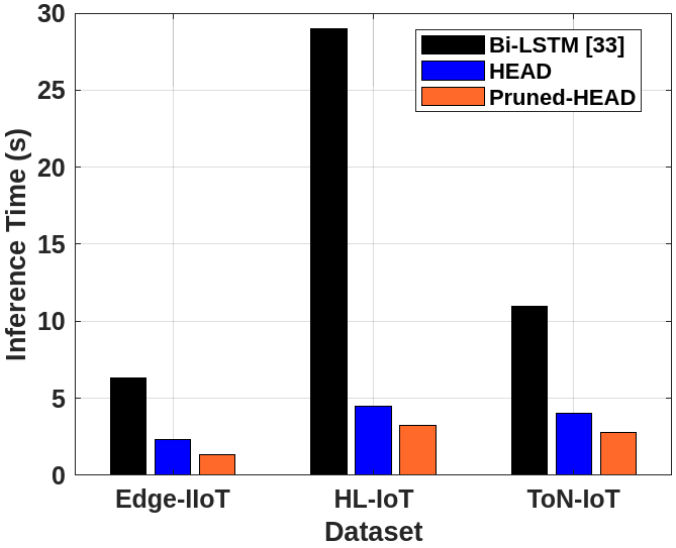


Fig. 6: System Inference Time Across Different Datasets

TABLE VII: Performance of HEAD System on Raspberry Pi 4 Model B

Dataset	Accuracy	Inference Time (s)	Memory Usage (MB)
Edge-IIoT	0.98	0.006	1272
HL-IoT	0.93	0.012	1436
ToN-IoT	0.96	0.010	1430

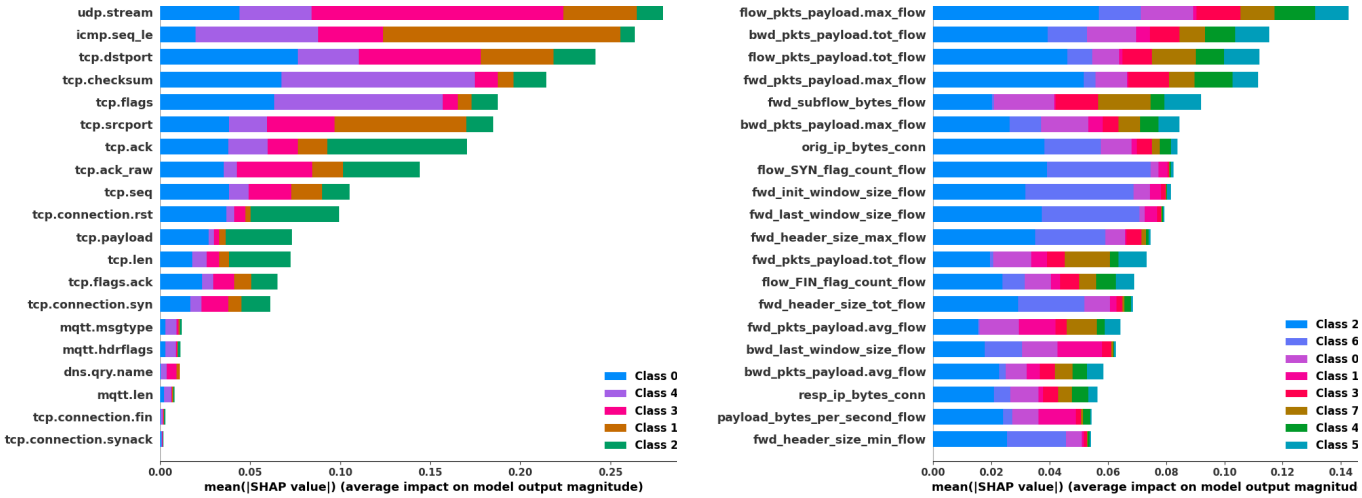
to 30–40% pruning, indicating that moderate pruning removes redundant neurons without affecting detection accuracy. For the Edge-IIoT dataset, 30% pruning preserves 0.99 F1 with lower complexity; for HL-IoT, 30–40% maintains 0.93 F1; and for ToN-IoT, 40% achieves 0.98 F1 before degradation begins. Therefore, 40% pruning is selected as the optimal configuration in the final HEAD system, providing a balance between accuracy, efficiency, and computational cost.

Fig. 5 shows the accuracy of various systems, including LSTM [16], Bi-LSTM [33], DEEPShield [13], 1D-CNN [36] and the HEAD system with and without pruning, which are evaluated on different datasets. On the Edge-IIoTSet dataset, the HEAD and Pruned-HEAD systems achieve an accuracy of 0.99, outperforming DEEPShield [13], 1D-CNN [36], Bi-LSTM [33], and LSTM [16] by up to 5%, 6%, 9%, and 8%, respectively. Similarly, on the HL-IoT dataset, the HEAD system reaches an accuracy of 0.96, surpassing DEEPShield [13], 1D-CNN [36], Bi-LSTM [33], and LSTM [16] by 2 to 7%. For the ToN-IoT dataset, the HEAD system again achieves the highest accuracy of 0.98, exceeding DEEPShield [13], 1D-CNN [36], Bi-LSTM [33], and LSTM [16]. The LSTM and Bi-LSTM systems apply DL without feature selection, limiting their ability to capture key patterns in heterogeneous IoT traffic. DEEPShield [13] improves performance using a CNN-LSTM ensemble model, and 1D-CNN [36] applies convolutional operations for spatial extraction, but both lack an optimized feature refinement stage. In contrast, the HEAD system uses a two-phase approach of first selecting important features with SHAP and then refining them with LIME.

Table VI shows the performance metrics of various DL systems, including LSTM [16], Bi-LSTM [33], DEEPShield [13], 1D-CNN [36], HEAD, and Pruned-HEAD, which are evalu-

Authorized licensed use limited to: University of Waterloo. Downloaded on March 19,2026 at 17:04:48 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2026.3653260



(a) Edge-IIoT Dataset

(b) HL-IoT Dataset

Fig. 7: SHAP Summary Plot for Different Classes in Various Datasets

TABLE VIII: Top Selected Important Features for Each Dataset

Rank	HL-IoT	ToN-IoT	Edge-IIoT
1	*fwd_subflow_bytes_flow*	*dns_rcode*	*icmp.seq_le*
2	*bwd_PUSH_flag_count_flow*	*dns_qclass*	*tcp.dstport*
3	*down_up_ratio_flow*	*dns_qtype*	*udp.stream*
4	*fwd_header_size_tot_flow*	*dst_port*	*tcp.checksum*
5	*flow_pkts_payload.std_flow*	*src_bytes*	*dns.qry.name*
6	*bwd_pkts_payload.avg_flow*	*duration*	*tcp.seq*
7	*fwd_init_window_size_flow*	*src_pkts*	*mqtt.msgtype*
8	*fwd_header_size_max_flow*	*src_ip_bytes*	*tcp.connection.syn*
9	*bwd_pkts_payload.tot_flow*	*dst_ip_bytes*	*http.content.length*
10	*bwd_iat.max_flow*	*src_port*	*arp.opcode*
11	*bwd_pkts_payload.std_flow*	*dst_pkts*	*mqtt.conflag.cleansess*
12	*fwd_pkt.tot_flow*	*dst_bytes*	*arp.hw.size*
13	*bwd_data_pkts_tot_flow*	*missed_bytes*	*tcp.flags.ack*
14	*flow_pkts_payload.max_flow*	*http_version*	*http.response*
15	*bwd_subflow_bytes_flow*	*http_trans_depth*	
16	*fwd_pkts_payload.avg_flow*	*http_status_code*	
17	*fwd_pkts_payload.max_flow*	*http_response_body_len*	
18	*fwd_last_window_size_flow*		
19	*fwd_iat.std_flow*		
20	*bwd_init_window_size_flow*		
21	*payload_bytes_per_second_flow*		
22	*bwd_last_window_size_flow*		
23	*bwd_pkts_per_sec_flow*		
24	*bwd_header_size_min_flow*		
25	*fwd_subflow_pkts_flow*		
26	*fwd_pkts_payload.std_flow*		

ated on the Edge-IIoT, HL-IoT, and ToN-IoT datasets. Among these, the HEAD system outperforms others in Pr, Rc, and F1 across all datasets. On the Edge-IIoT dataset, the HEAD system achieves an average F1 of 0.96, compared to the DEEPShield [13] system at 0.93, Bi-LSTM [33] at 0.91, LSTM [16] at 0.91, and 1D-CNN [36] at 0.90, showing improvements of approximately 3.2%, 5.5%, 5.5%, and 6.6%, respectively. The Pruned-HEAD system closely follows with an F1 of 0.96, indicating strong retention of accuracy after pruning. For the HL-IoT dataset, the HEAD system maintains an average F1 of 0.96, compared to DEEPShield [13] at 0.93, Bi-LSTM [33] at 0.91, LSTM at 0.91, and 1D-CNN at 0.84, with improvements of 3.2%, 5.5%, 5.5%, and 12.5%, respectively. In particular, the HEAD system achieves a near-perfect F1 of 0.99 for class 2 and 0.96 for Class 6. The Pruned-HEAD system also achieves a comparable F1-score of 0.95, showing its reliability in classification. On the ToN-IoT dataset, the HEAD system achieves an average F1 of 0.96, outperforming DEEPShield (0.92), Bi-LSTM (0.89), LSTM (0.89), and 1D-CNN (0.93) by approximately 4% to 8%. The Pruned-HEAD system achieves 0.95, nearly matching the performance of the original HEAD system. Overall, the Pruned-HEAD system achieves nearly equal accuracy compared to the HEAD system across all datasets, but with lower inference time, making it more suitable for deployment in resource-constrained networks.

Fig. 6 shows the inference time comparison of three systems: Bi-LSTM [33], HEAD and Pruned-HEAD across the Edge-IIoT, HL-IoT and ToN-IoT datasets. The Bi-LSTM [33] system records the highest inference time on all datasets, approximately 6.3 seconds for Edge-IIoT, 29 seconds for HL-IoT, and 11 seconds for ToN-IoT. In comparison, the HEAD

Authorized licensed use limited to: University of Waterloo. Downloaded on March 19,2026 at 17:04:48 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2026.3653260

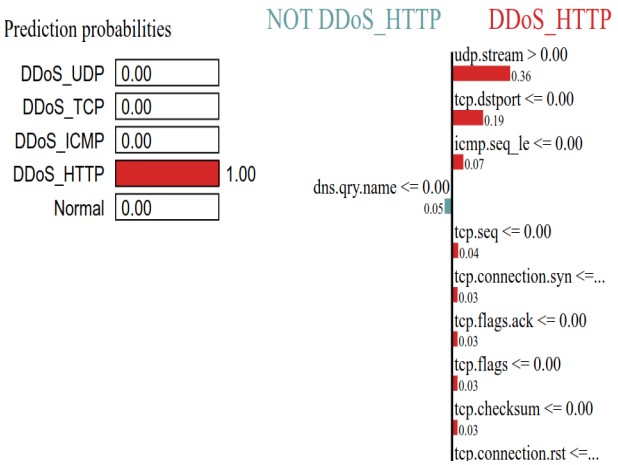


Fig. 8: LIME Explanation for DDoS_HTTP in the Edge-IIoTSet Dataset

system achieves lower inference times, around 2.35 seconds, 4.5 seconds, and 4 seconds on the respective datasets. The Pruned-HEAD system achieves the best results by further reducing the inference time to approximately 1.32 seconds for Edge-IIoT, 3.23 seconds for HL-IoT, and 2.79 seconds for ToN-IoT. On average, the Pruned-HEAD system is about 2 times faster than the HEAD system and more than 4 times faster than the Bi-LSTM system. These results show that the Pruned-HEAD system provides faster inference while still maintaining good accuracy.

The HEAD system deploys on a Raspberry Pi 4 Model B with 4 GB of RAM and a 1.5 GHz quad-core ARM Cortex-A72 processor to verify its suitability for edge deployment. Offline traffic data from each dataset is processed in batches of 64 samples to simulate real on-device inference conditions. Table VII shows the accuracy, inference time, and memory usage obtained for the Edge-IIoT, HL-IoT, and ToN-IoT datasets. The HEAD system achieves an accuracy of 0.98 on Edge-IIoT, 0.93 on HL-IoT, and 0.96 on ToN-IoT, showing consistent detection performance across different networks. The inference time remains below 0.012 seconds for all datasets, indicating that the lightweight model can make near real-time predictions on constrained hardware. Memory usage ranges between 1.27 GB and 1.43 GB, which remains well within the available resources of the Raspberry Pi. These results show that the optimized and pruned-HEAD system is efficient, scalable, and suitable for real-world use on low-power IIoT edge devices.

Fig. 7 shows the SHAP summary plots for different classes in the Edge-IIoT and HL-IoT datasets. The Edge-IIoT dataset represents an industrial network with high-volume DDoS attacks. The HL-IoT dataset includes both high- and low-volume attacks, providing insight into the detection of stealthy low-rate and large-scale floods. In both plots, the x-axis shows the mean absolute SHAP value, which represents the average influence of each feature on the model's predictions. Higher SHAP values represent greater influence on classification and help identify the most critical features for DDoS attack detection. The bars represent feature contributions across different attack classes, where each colour corresponds to a specific class. A longer bar indicates that the feature has a greater impact on the predictions of multiple types of attack. For the Edge-IIoT dataset (Fig. 7(a)), features such as *udp.stream*, *icmp.seq_le*, and *tcp.dstport* have the highest SHAP values, which indicates their importance in detecting DDoS attacks. These features play an important role in identifying anomalies in UDP and ICMP traffic, which are common targets in volumetric DDoS attacks. Other key features, such as *tcp.flags* and *tcp.seq*, help detect irregular TCP connection behaviours linked to SYN flood attacks. For the HL-IoT dataset (Fig. 7(b)), features such as *flow_pkts_payload.max_flow*, *bwd_pkts_payload.tot_flow*, and *fwd_pkts_payload.max_flow* dominate the SHAP summary plot, which shows their importance in both high- and low-volume attack detection. The presence of multiple flow-based features indicates that both volumetric floods and stealthy low-rate attacks can be effectively distinguished using traffic flow patterns. Features like *flow_SYN_flag_count_flow* and *flow_FIN_flag_count_flow* also play a key role in identifying low-volume TCP-based attacks, where attackers manipulate packet flags to evade detection.

Table VIII shows the top-ranked features for each dataset after applying the combined importance of SHAP-LIME. The HL-IoT dataset focuses on flow-based features that provide effective detection of both high- and low-volume attacks. The ToN-IoT dataset emphasizes DNS-related features like *dns_rcode* and *dns_qclass*, which provide the importance of query-response behaviours in attack detection. The Edge-IIoT dataset focuses on transport and network layer parameters that capture large-scale volumetric patterns common in industrial networks. These selected features improve classification performance by making the model easier to interpret and more effective in different attack situations.

The HEAD system reduces complexity through both feature selection and neuron pruning. In the HL-IoT dataset, the SHAP–LIME process reduces the input space from 91 features to a final set of 26. This reduces the trainable parameters of the model from 8,824 to 4,664, which is a 47% decrease. After this, SHAP-based pruning removes 40% of the neurons from the first dense layer, further reducing the parameter count to 3,139. This step provides an additional 33% reduction. Overall, the combination of feature selection and pruning reduces the total trainable parameters by about 64% compared to the full model. This reduction helps the HEAD system maintain fast inference and lower computational load.

Fig. 8 shows the LIME visualization for a sample instance from the Edge-IIoT dataset belonging to Class 0 (DDoS HTTP). As LIME is a local explanation technique, this graph offers a deeper understanding of the model's decision-making for this instance. The left side shows the prediction probabilities for all classes, with the DDoS HTTP class having a confidence score of 1.00, reflecting the model's high confidence in its prediction. On the right, the top contributing features are displayed, ranked according to their impact on the decision. The feature *udp.stream* >0.00 makes the strongest positive contribution, pushing the prediction towards the DDoS HTTP class. Other features, such as *tcp.dstport* $<= 0.00$ and *icmp.seq_le* $<= 0.00$, also have a positive impact by capturing patterns typical of HTTP-based DDoS traffic. Conversely, features like *dns.qry.name* $<= 0.00$ slightly lowers the probability

Authorized licensed use limited to: University of Waterloo. Downloaded on March 19,2026 at 17:04:48 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2026.3653260

of classifying the instance as DDoS HTTP.

Fig. 9 shows the user interface of the HEAD system running on localhost. The interface allows administrators and researchers to visualize and analyze how key features contribute to DDoS attack detection. The interface allows users to select an analysis method from SHAP, LIME, or a hybrid approach, and upload a dataset for preprocessing and analysis. The data cleaning and preprocessing section displays the class distributions before and after applying SMOTE. The plot in the left bar shows how SMOTE balances the dataset by oversampling minority classes for equal representation. The SHAP summary graph on the right highlights the most important features that influence the prediction of the model, with colour-coded bars that indicate the direction and magnitude of their impact. This interface improves the interpretability of the detection process and allows users to explore feature importance, identify misclassifications, and refine defence strategies against DDoS attacks interactively.

D. Discussion

- The inference is performed on pre-collected traffic batches rather than continuous packet streams. This means it does not consider queuing delays or buffering effects. Such factors appear during live traffic capture in high-speed industrial networks.
- The current setup runs only on CPU resources without any hardware acceleration. This limits throughput when handling multiple traffic flows in large IIoT environments.
- The hyperparameters (α, β, and τ) are adjusted experimentally by varying their values within specific ranges. Their impact is observed on F1-score, feature count, and computational time. The selected values ($\alpha = 0.7$, $\beta = 0.5$, $\tau = 0.005$) show the best balance between accuracy, feature compactness, and efficiency across all datasets.
- The HEAD system performs better than LSTM, Bi-LSTM, 1D-CNN, and DEEPShield across all datasets. This is because it uses a two-phase explainable feature refinement process. LSTM and Bi-LSTM miss optimized feature selection, and 1D-CNN ignores time-based patterns. DEEPShield improves modelling but still lacks transparency and optimization. HEAD, however, combines explainability, pruning, and DL for higher accuracy, better efficiency, and clearer interpretation.
- The HEAD system uses a gradient-based attribution approach to measure neuron importance in the first hidden layer. SHAP does not directly assess deeper layers, but pruning less important neurons early reduces unnecessary activity later in the network, leading to better generalization and efficiency.

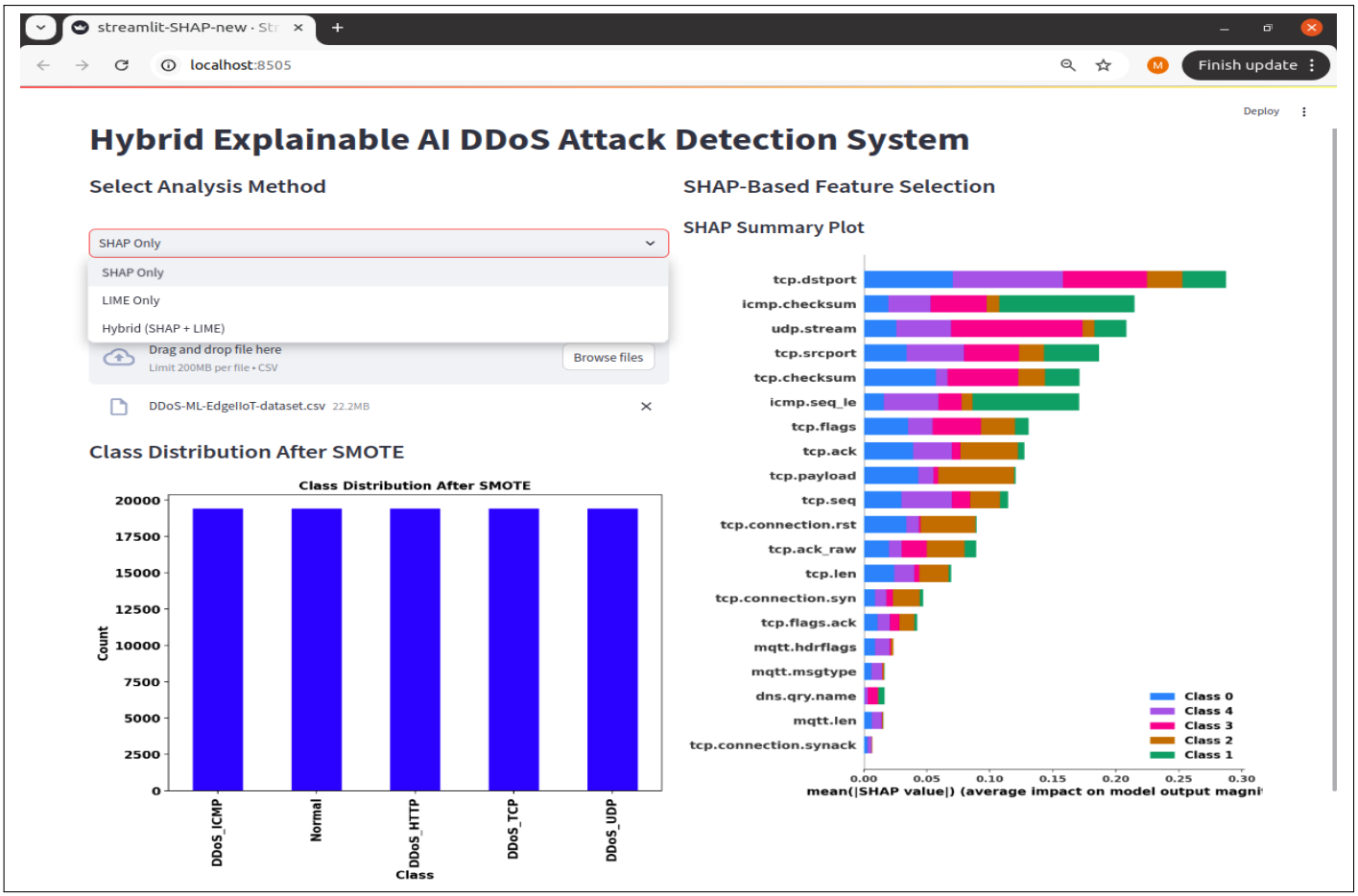


Fig. 9: Screenshot of the Streamlit-based user interface of the HEAD system

Authorized licensed use limited to: University of Waterloo. Downloaded on March 19,2026 at 17:04:48 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2026.3653260

- The HEAD system design supports deployment on multiple edge nodes. Each node runs the same lightweight feature extraction and detection modules and processes its own incoming traffic independently. In a multi-edge setup, nodes perform real-time inference locally and exchange only alert metadata rather than raw traffic. This reduces communication overhead and keeps the system scalable in distributed IIoT networks.

V. Conclusion

This paper proposed the Hybrid and Explainable AI for DDoS (HEAD) attack detection system designed to identify high- and low-volume DDoS attacks in IIoT networks. The HEAD system used a two-phase feature selection process that integrated SHAP for global feature importance and LIME for local interpretability using the Random Forest model. A feedforward DNN was trained on the selected features, followed by SHAP-based neuron importance analysis to prune low-impact neurons. This post-training optimization reduced the latency of inference without compromising detection accuracy. The HEAD system with the original and pruned DL models outperformed existing methods. On multiple datasets, the HEAD system achieves higher precision, recall, and F1 scores. The SHAP plots showed key features that influenced attack detection, such as packet size, payload, connection duration, and protocol behaviour. The LIME plot provided individual prediction explanations by focusing on the contribution of features. In addition, the HEAD system included a user interface that allowed administrators to visualize predictions, the importance of features, and system decisions. Future work will focus on improving the hybrid feature selection strategy to further reduce computational overhead without affecting detection performance. It will also aim to adapt the HEAD system for real-time use in dynamic and large-scale IIoT networks. Future work will also aim to make the system more robust against changing DDoS attack patterns. This can be achieved by adding adaptive learning methods to handle concept drift. The work will also explore lightweight model compression techniques such as pruning and quantization to support efficient deployment on edge devices.

Acknowledgment

This work was supported in part by the Natural Sciences and Engineering Research Council of Canada (NSERC) under Grant RGPIN-2019-06060.

References

[1] A. Zainudin, L. A. C. Ahakonye, R. Akter, D.-S. Kim, and J.-M. Lee, "An efficient hybrid-dnn for ddos detection and classification in software-defined iiot networks," *IEEE Internet of Things Journal*, vol. 10, no. 10, pp. 8491–8504, 2022.

[2] S. Velliangiri, N. G. B. Amma, and N.-K. Baik, "Detection of dos attacks in smart city networks with feature distance maps: A statistical approach," *IEEE Internet of Things Journal*, vol. 10, no. 21, pp. 18 853–18 860, 2023.

[3] Y. P. Tsang, C.-H. Wu, H. Y. Lam, K. L. Choy, and G. T. Ho, "Integrating internet of things and multi-temperature delivery planning for perishable food e-commerce logistics: A model and application," *International Journal of Production Research*, vol. 59, no. 5, pp. 1534–1556, 2021.

[4] M. Shafiq, Z. Tian, A. K. Bashir, X. Du, and M. Guizani, "Corrauc: a malicious bot-iot traffic detection method in iot network using machine learning techniques," *IEEE Internet of Things Journal*, vol. 8, no. 5, pp. 3242–3254, 2020.

[5] T. Qiu, J. Chi, X. Zhou, Z. Ning, M. Atiquzzaman, and D. O. Wu, "Edge computing in industrial internet of things: Architecture, advances and challenges," *IEEE Communications Surveys & Tutorials*, vol. 22, no. 4, pp. 2462–2488, 2020.

[6] M. Zolanvari, M. A. Teixeira, L. Gupta, K. M. Khan, and R. Jain, "Machine learning-based network vulnerability analysis of industrial internet of things," *IEEE IoT Journal*, vol. 6, no. 4, pp. 6822–6834, 2019.

[7] H. Kheddar, Y. Himeur, and A. I. Awad, "Deep transfer learning for intrusion detection in industrial control networks: A comprehensive review," *Journal of Network and Computer Applications*, vol. 220, p. 103760, 2023.

[8] W. Mao, Z. Zhao, Z. Chang, G. Min, and W. Gao, "Energy-efficient industrial internet of things: Overview and open issues," *IEEE transactions on industrial informatics*, vol. 17, no. 11, pp. 7225–7237, 2021.

[9] M. F. Saiyed and I. Al-Anbagi, "A genetic algorithm-and t-test-based system for ddos attack detection in iot networks," *IEEE Access*, vol. 12, pp. 25 623–25 641, 2024.

[10] S. Chaudhary and P. K. Mishra, "Ddos attacks in industrial iot: A survey," *Computer Networks*, p. 110015, 2023.

[11] M. F. Saiyed and I. Al-Anbagi, "Flow and unified information-based ddos attack detection system for multi-topology iot networks," *Internet of Things*, vol. 24, p. 100976, 2023.

[12] R. Bocu and M. Iavich, "Enhanced detection of low-rate ddos attack patterns using machine learning models," *Journal of Network and Computer Applications*, vol. 227, p. 103903, 2024.

[13] M. F. Saiyed and and I. Al-Anbagi, "Deep ensemble learning with pruning for ddos attack detection in iot networks," *IEEE Transactions on Machine Learning in Communications and Networking*, vol. 2, pp. 596–616, 2024.

[14] K. Doshi, Y. Yilmaz, and S. Uludag, "Timely detection and mitigation of stealthy ddos attacks via iot networks," *IEEE Transactions on Dependable and Secure Computing*, vol. 18, no. 5, pp. 2164–2176, 2021.

[15] M. Lefoane, I. Ghafir, S. Kabir, and I.-U. Awan, "Internet of things botnets: A survey on artificial intelligence based detection techniques," *Journal of Network and Computer Applications*, vol. 236, p. 104110, 2025.

[16] A. A. Alashhab, M. S. M. Zahid, A. Muneer, and M. Abdullahi, "Low-rate ddos attack detection using deep learning for sdn-enabled iot networks," *International Journal of Advanced Computer Science and Applications*, vol. 13, no. 11, 2022.

[17] M. Malik, M. Dutta *et al.*, "Feature engineering and machine learning framework for ddos attack detection in the standardized internet of things," *IEEE Internet of Things Journal*, vol. 10, no. 10, pp. 8658–8669, 2023.

[18] F. Taher, M. Abdel-salam, M. Elhoseny, and I. El-hasnony, "Reliable machine learning model for iiot botnet detection," *IEEE Access*, vol. 11, pp. 34 019–34 031, 2023.

[19] S. M. Kasongo, "An advanced intrusion detection system for iiot based on ga and tree based algorithms," *IEEE Access*, vol. 9, pp. 113 199–113 212, 2021.

[20] M. F. Saiyed, I. Al-Anbagi, and M. S. Hossain, "Interactive and explainable optimized learning for ddos detection in consumer iot networks," *IEEE Transactions on Consumer Electronics*, pp. 1–1, 2024.

[21] O. Arreche, T. R. Guntur, J. W. Roberts, and M. Abdallah, "E-xai: Evaluating black-box explainable ai frameworks for network intrusion detection," *IEEE Access*, 2024.

[22] Z. Zhang, H. Al Hamadi, E. Damiani, C. Y. Yeun, and F. Taher, "Explainable artificial intelligence applications in cyber security: State-of-the-art in research," *IEEE Access*, vol. 10, pp. 93 104–93 139, 2022.

[23] D. Javeed, M. S. Saeed, I. Ahmad, P. Kumar, A. Jolfaei, and M. Tahir, "An intelligent intrusion detection system for smart consumer electronics network," *IEEE Transactions on Consumer Electronics*, vol. 69, no. 4, pp. 906–913, 2023.

[24] Y. Feng, J. Li, D. Sisodia, and P. Reiher, "On explainable and adaptable detection of distributed denial-of-service traffic," *IEEE Transactions on Dependable and Secure Computing*, 2023.

[25] R. K. Batchu and H. Seetha, "An integrated approach explaining the detection of distributed denial of service attacks," *Computer Networks*, vol. 216, p. 109269, 2022.

[26] M. Wang, K. Zheng, Y. Yang, and X. Wang, "An explainable machine learning framework for intrusion detection systems," *IEEE Access*, vol. 8, pp. 73 127–73 141, 2020.

Authorized licensed use limited to: University of Waterloo. Downloaded on March 19,2026 at 17:04:48 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2026.3653260

[27] B. Sharma, L. Sharma, C. Lal, and S. Roy, "Explainable artificial intelligence for intrusion detection in iot networks: A deep learning based approach," *Expert Systems with Applications*, vol. 238, p. 121751, 2024.

[28] B. Tag, N. Van Berkel, S. Verma, B. Z. H. Zhao, S. Berkovsky, D. Kaafar, V. Kostakos, and O. Ohrimenko, "Ddod: Dual denial of decision attacks on human-ai teams," *IEEE Pervasive Computing*, vol. 22, no. 1, pp. 77–84, 2023.

[29] M. A. Ferrag, O. Friha, D. Hamouda, L. Maglaras, and H. Janicke, "Edge-iiotset: A new comprehensive realistic cyber security dataset of iot and iiot applications for centralized and federated learning," *IEEE Access*, vol. 10, pp. 40 281–40 306, 2022.

[30] N. Moustafa, "Ton-iot dataset," Online, 2020, accessed: Jul. 15, 2023. [Online]. Available: https://research.unsw.edu.au/projects/toniot-datasets

[31] E. Gyamfi and A. D. Jurcut, "Novel online network intrusion detection system for industrial iot based on oi-svdd and as-elm," *IEEE Internet of Things Journal*, vol. 10, no. 5, pp. 3827–3839, 2023.

[32] N. Moustafa, E. Adi, B. Turnbull, and J. Hu, "A new threat intelligence scheme for safeguarding industry 4.0 systems," *IEEE Access*, vol. 6, pp. 32 910–32 924, 2018.

[33] P. TS and P. Shrinivasacharya, "Evaluating neural networks using bi-directional lstm for network ids (intrusion detection systems) in cyber security," *Global Transitions Proceedings*, vol. 2, no. 2, pp. 448–454, 2021, international Conference on Computing System and its Applications (ICCSA- 2021).

[34] U. Ahmed, Z. Jiangbin, A. Almogren, M. Sadiq, A. U. Rehman, M. Sadiq, and J. Choi, "Hybrid bagging and boosting with shap based feature selection for enhanced predictive modeling in intrusion detection systems," *Scientific Reports*, vol. 14, no. 1, p. 30532, 2024.

[35] Y. Ma, L. Liu, Z. Liu, F. Li, Q. Xie, K. Chen, C. Lv, Y. He, and F. Li, "A survey of ddos attack and defense technologies in multiaccess edge computing," *IEEE Internet of Things Journal*, vol. 12, no. 2, pp. 1428–1452, 2025.

[36] M. F. Saiyed and I. Al-Anbagi, "An intelligent intent-aware system for ddos attacks detection and mitigation in iot networks," *IEEE Internet of Things Journal*, 2025, under review.

[37] I. Hafeez, M. Antikainen, A. Ding, and S. Tarkoma, "Iot-keeper: Detecting malicious iot network activity using online traffic analysis at the edge," *IEEE Trans. on Network and Service Management*, vol. 17, no. 1, pp. 45–59, 2020.

[38] M. Saiyed and I. Al Anbagi, "Entropy and divergence-based ddos attack detection system in iot networks," in *2023 19th International Conference on WiMob*. IEEE, 2023, pp. 224–230.

[39] Y. Wei, J. Jang-Jaccard, A. Singh, F. Sabrina, and S. Camtepe, "Classification and explanation of distributed denial-of-service (ddos) attack detection using machine learning and shapley additive explanation (shap) methods," *arXiv preprint arXiv:2306.17190*, 2023.

[40] D. Javeed, T. Gao, P. Kumar, and A. Jolfaei, "An explainable and resilient intrusion detection system for industry 5.0," *IEEE Transactions on Consumer Electronics*, vol. 70, no. 1, pp. 1342–1350, 2024.

[41] S. S. Shafin, "An explainable feature selection framework for web phishing detection with machine learning," *Data Science and Management*, 2024.

[42] N. Yoon and H. Kim, "Detecting ddos based on attention mechanism for software-defined networks," *Journal of Network and Computer Applications*, vol. 230, p. 103928, 2024.

[43] N. Wei, L. Yin, J. Tan, C. Ruan, C. Yin, Z. Sun, and X. Luo, "An autoencoder-based hybrid detection model for intrusion detection with small-sample problem," *IEEE Transactions on Network and Service Management*, vol. 21, no. 2, pp. 2402–2412, 2024.

[44] J. Mao, Z. Wei, B. Li, R. Zhang, and L. Song, "Toward ever-evolution network threats: A hierarchical federated class-incremental learning approach for network intrusion detection in iiot," *IEEE Internet of Things Journal*, vol. 11, no. 18, pp. 29 864–29 877, 2024.

[45] K. Adamczewski, Y. Li, and L. Van Gool, "Shapley pruning for neural network compression," *arXiv preprint arXiv:2407.15875*, 2024.

[46] F. Merkle, D. Weber, P. Schöttle, S. Schlögl, and M. Nocker, "Less is more: The influence of pruning on the explainability of cnns," *IEEE Access*, 2025.

[47] K. Saurabh, T. Kumar, U. Singh, O. Vyas, and R. Khondoker, "Nfdlm: A lightweight network flow based deep learning model for ddos attack detection in iot domains," in *2022 IEEE World AI IoT Congress (AIIoT)*. IEEE, 2022, pp. 736–742.

[48] A. Y. Zalozhnev and V. N. Ginz, "Industry 4.0: Underlying technologies. industry 5.0: Human-computer interaction as a tech bridge from industry 4.0 to industry 5.0," in *2023 9th International Conference on Web Research (ICWR)*. IEEE, 2023, pp. 232–236.

[49] S. Tabassum, N. Parvin, N. Hossain, A. Tasnim, R. Rahman, and M. I. Hossain, "Iot network attack detection using xai and reliability analysis," in *2022 25th International Conference on Computer and Information Technology (ICCIT)*. IEEE, 2022, pp. 176–181.

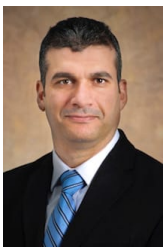
[50] J. Zhou, Y. Cheng, and C. Dai, "From the perspective of explainable machine learning: A student feature selection strategy based on the geometric mean of feature importance and robustness," in *Proceedings of the 2024 International Conference on Computer and Multimedia Technology*, 2024, pp. 499–503.

[51] R. Doriguzzi-Corin, S. Millar, S. Scott-Hayward, J. Martinez-del Rincon, and D. Siracusa, "Lucid: A practical, lightweight deep learning solution for ddos attack detection," *IEEE Transactions on Network and Service Management*, vol. 17, no. 2, pp. 876–889, 2020.

[52] J. Goldwasser and G. Hooker, "Statistical significance of feature importance rankings," in *The 41st Conference on Uncertainty in Artificial Intelligence*, 2024.

[53] K. Khadka, S. Shree, Y. Lei, R. N. Kacker, and D. R. Kuhn, "Assessing the degree of feature interactions that determine a model prediction," in *2024 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*. IEEE, 2024, pp. 180–187.

Makhduma F Saiyed received her Ph.D. in Electronic Systems Engineering from the University of Regina, Canada, in 2025. She served as a Postdoctoral Fellow at the University of Regina from March to June 2025. Currently, she is an Assistant Professor in the Department of Computer Science at Trent University, Oshawa, Ontario, Canada. She is also a registered Engineer-in-Training (EIT) with APEGS. She has over 10 years of teaching experience and has published numerous research papers. Her current research interests include network security, the Internet of Things (IoT), machine learning, deep neural networks, game theory, explainable AI, and large language models.



Irfan Al-Anbagi (senior member IEEE) received his Ph.D. in Electrical and Computer Engineering from the University of Ottawa in October 2013. From 2013 to 2015, he was a postdoctoral fellow and the product development manager of the SecCharge Project at the University of Ottawa. He is a Professor with the Faculty of Engineering and Applied Science at the University of Regina and an Adjunct Professor in the Department of Electrical and Computer Engineering, College of Engineering at the University of Saskatchewan. He is registered as a professional engineer with the Association of Professional Engineers and Geoscientists of Saskatchewan (APEGS) and Professional Engineers Ontario (PEO). His research interests include security and reliability in cyber-physical systems, Internet of Things (IoT) systems, and edge computing.

Authorized licensed use limited to: University of Waterloo. Downloaded on March 19,2026 at 17:04:48 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.